

Guia de Setup — Pajeu Ticket

Instruções para clonar e rodar **API**, **Frontend** e **PostgreSQL** na máquina local.

1. Pré-requisitos

Ferramenta	Versão mínima	Verificar
Git	qualquer recente	<code>git --version</code>
Java JDK	21	<code>java -version</code>
Maven	3.9+	<code>mvn -version</code>
Node.js	20+ (recomendado 22)	<code>node -v</code>
npm	10+	<code>npm -v</code>
PostgreSQL	14+	<code>psql --version</code>

Instalação rápida (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y git openjdk-21-jdk maven nodejs npm postgresql postgresql-contrib
```

No Windows: instale JDK 21 (adoptium.net), Maven, Node.js LTS e PostgreSQL.

2. Clonar os repositórios

```
mkdir -p ~/projetos && cd ~/projetos

git clone https://github.com/brunorah00/Pajeu-Ticket-API.git
git clone https://github.com/brunorah00/Pajeu-Ticket-Web.git
```

3. Configurar o PostgreSQL

3.1 Criar usuário e banco (Linux)

Substitua `seu_usuario` e `sua_senha` pelos dados que o colaborador quiser usar:

```
sudo -u postgres psql
```

No `psql` :

```
CREATE USER seu_usuario WITH PASSWORD 'sua_senha' CREATEDB;  
CREATE DATABASE cinepajeu OWNER seu_usuario ENCODING 'UTF8';  
GRANT ALL PRIVILEGES ON DATABASE cinepajeu TO seu_usuario;  
\q
```

Testar conexão:

```
PGPASSWORD=sua_senha psql -h localhost -U seu_usuario -d cinepajeu -c "SELECT 1;"
```

3.2 Script automático (alternativa)

Na pasta da API:

```
cd ~/projetos/Pajeu-Ticket-API  
DB_USERNAME=seu_usuario DB_PASSWORD=sua_senha ./scripts/create-database.sh
```

Se pedir sudo, edite `scripts/create-database.sql` trocando o usuário padrão pelo dele e rode:

```
sudo -u postgres psql -f scripts/create-database.sql
```

4. Configurar a API (Backend)

```
cd ~/projetos/Pajeu-Ticket-API  
cp .env.example .env
```

Edite o `.env` :

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=cinepajeu
DB_USERNAME=seu_usuario
DB_PASSWORD=sua_senha

JWT_SECRET=uma-chave-secreta-longa-e-aleatoria-aqui

FRONTEND_URL=http://localhost:3000
```

Opcional (login Google/Facebook e e-mail):

```
GOOGLE_CLIENT_ID=
FACEBOOK_APP_ID=
FACEBOOK_APP_SECRET=
MAIL_HOST=
MAIL_PORT=587
MAIL_USERNAME=
MAIL_PASSWORD=
MAIL_FROM=noreply@cinesaojose.com
```

Sem SMTP, links de recuperação de senha aparecem no **log da API**.

4.1 Dados iniciais (seed) — rodar uma vez

O Spring **não** reinsere filmes a cada restart. Para popular admin, filmes e produtos de exemplo:

```
PGPASSWORD=sua_senha psql -h localhost -U seu_usuario -d cinepajeu \
-f src/main/resources/import.sql
```

4.2 Subir a API

```
chmod +x run-dev.sh
./run-dev.sh
```

Aguarde no log: **Tomcat started on port 8080**.

Alternativa:

```
mvn spring-boot:run
```

- API: `http://localhost:8080`
- Swagger: `http://localhost:8080/swagger-ui.html`

Na **primeira subida**, o Hibernate cria/atualiza as tabelas e o `schema-patch.sql` aplica migrações idempotentes.

5. Configurar o Frontend (Web)

Em **outro terminal**:

```
cd ~/projetos/Pajeu-Ticket-Web
npm install
cp .env.example .env.local
```

Edite `.env.local` (mínimo necessário):

```
NEXT_PUBLIC_API_URL=http://localhost:8080
```

Opcional (login social):

```
NEXT_PUBLIC_GOOGLE_CLIENT_ID=mesmo-id-do-google
NEXT_PUBLIC_FACEBOOK_APP_ID=mesmo-id-do-facebook
```

5.1 Subir o frontend

```
npm run dev
```

- Site: `http://localhost:3000`

Se der erro de cache (`ENOENT` em `.next`):

```
rm -rf .next
npm run dev
```

6. Ordem de execução (resumo)

Terminal	Comando	URL
1 — PostgreSQL	<code>serviço ativo (sudo systemctl start postgresql)</code>	porta 5432
2 — API	<code>cd Pajeu-Ticket-API && ./run-dev.sh</code>	http://localhost:8080
3 — Web	<code>cd Pajeu-Ticket-Web && npm run dev</code>	http://localhost:3000

7. Login para testes

Após rodar o `import.sql` :

Perfil	Login	Senha
Admin (painel)	<code>pajeuticket</code>	<code>Ticket202!!</code>

Rotas principais:

- Site: <http://localhost:3000>
- Painel: <http://localhost:3000/painel/dashboard>
- Bomboniere: <http://localhost:3000/bomboniere>
- Comprar ingresso: <http://localhost:3000/ingressos/comprar>

8. Problemas comuns

Porta 8080 em uso (API)

```
fuser -k 8080/tcp  
./run-dev.sh
```

Porta 3000 em uso (Front)

```
fuser -k 3000/tcp  
npm run dev
```

API não conecta no banco

- Conferir `.env` (`DB_NAME` , `DB_USERNAME` , `DB_PASSWORD`)
- Testar: `PGPASSWORD=... psql -h localhost -U seu_usuario -d cinepajeu -c "SELECT 1;"`

Front não fala com a API

- API rodando em 8080
- `.env.local` com `NEXT_PUBLIC_API_URL=http://localhost:8080`
- Reiniciar o `npm run dev` após alterar `.env.local`

Filmes duplicados após restart

Não deve ocorrer na versão atual. O `import.sql` não roda mais automaticamente.

Cartazes / uploads

Imagens ficam em `Pajeu-Ticket-API/uploads/`. Essa pasta **não vai no Git**; cada dev faz upload pelo painel.

9. Checklist final

- ☐ PostgreSQL rodando
- ☐ Banco `cinepajeu` criado
- ☐ `.env` da API configurado
- ☐ `import.sql` executado (opcional, para ter admin e dados de exemplo)
- ☐ API em `http://localhost:8080` (Swagger abre)
- ☐ `.env.local` do Web com `NEXT_PUBLIC_API_URL`
- ☐ Front em `http://localhost:3000`
- ☐ Login no painel com `pajeuticket` / `Ticket202!!`

